Computability – Topic 11

Yonghao Lee

March 12, 2026

Contents

1 The Hamiltonian Path Problem **2**

2 UHAMPATH **7**

3 Bounded NFA Non-Universality **9**

1 The Hamiltonian Path Problem

Recall that the Hamiltonian path problem asks whether the input graph contains a path from s to t that goes through every node exactly once.

Remark 1.1: The General Pattern of NP-Completeness Proofs

To show that a problem B is NP-complete, we always follow the same recipe:

1. **Show** $B \in \mathrm{NP}$: Provide a polynomial-time verifier with an appropriate certificate.
2. **Show** $A \leq_P B$ **for some known NP-complete problem** A: Construct a polynomial-time *reduction*—a function f that transforms any instance of A into an instance of B such that YES maps to YES and NO maps to NO.

Crucially, a reduction translates **problem structure**, not solutions. The function f is built from the internal structure of the input instance (e.g., which variables appear in which clauses) without knowing whether a solution exists. The solutions only appear in the **correctness proof**, where we show:

- If a solution to A exists, then the translated instance of B also has a solution.
- If a solution to B exists, then the original instance of A also has a solution.

The instances themselves are the *questions* (a formula, a graph), not the answers (an assignment, a path).

Theorem 1.1

HAMPATH is NP-complete.

Remark 1.2: What are the instances?

- **Instance of** 3SAT: A specific 3-CNF formula ϕ, e.g., $\phi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_3)$. The question: does there *exist* a satisfying assignment?
- **Instance of** HAMPATH: A specific directed graph G with specific nodes s and t. The question: does there *exist* a Hamiltonian path from s to t?

The reduction is a function f : formulas $\to$ graphs, mapping $\phi \mapsto (G, s, t)$. It maps a **question to a question**, not an answer to an answer. Given any specific ϕ, the output is a fully determined graph with a fixed number of nodes and edges—every node and edge is determined by the structure of ϕ alone.

Proof. We first show that HAMPATH $\in$ NP by presenting a polynomial-time verifier. Given a directed graph G with designated nodes s and t, the certificate is a sequence of n nodes. The verifier checks that the sequence starts at s, ends at t, visits every node exactly once, and that each consecutive pair is a valid edge in G. This can be done in polynomial time.

It remains to show that 3SAT $\leq_P$ HAMPATH, which by transitivity (since 3SAT is NP-complete) implies that every language in NP is polynomial-time reducible to HAMPATH.

For each 3-CNF formula ϕ, we construct a directed graph G with two distinguished nodes s and t such that a Hamiltonian path from s to t exists in G if and only if ϕ is satisfiable.

Setup. Let ϕ be a 3-CNF formula with k clauses:

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$$

where each a_i, b_i, c_i is a literal x_j or $\overline{x_j}$, and the three literals in each clause correspond to three distinct variables. Let $x_1, x_2, \ldots, x_l$ be the l variables of ϕ.

Construction of G.

The idea is to build a graph whose structure mirrors the logical structure of ϕ: variables become gadgets with two traversal directions (corresponding to TRUE/FALSE), clauses become standalone nodes that must be visited, and the edges are arranged so that "visiting every node" in the graph corresponds exactly to "satisfying every clause" in the formula.

Variable gadgets. We represent each variable x_i with a diamond-shaped structure. The diamond has a **top node** (the choice point) and a **bottom node** (the convergence point) connected by a horizontal row of $3k + 1$ nodes. The top node has edges to both ends of the row, forcing the path to choose a direction. Both ends of the row have edges into the bottom node, funneling the path back to a single node regardless of which direction was chosen.

The diamonds are chained vertically: s connects to the top of the first diamond, each diamond's bottom node connects to the next diamond's top node, and the last diamond's bottom node connects to t. The horizontal row can be traversed in two directions: left-to-right corresponds to setting x_i = TRUE, and right-to-left corresponds to setting x_i = FALSE.

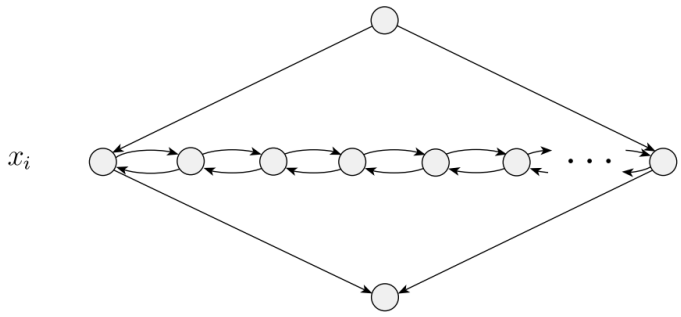

Figure 1: The variable x_i as a diamond structure.

Clause nodes. We represent each clause c_j of ϕ as a single standalone node, separate from all the diamond structures. The following figure depicts the global structure of G, showing all elements and their relationships *except* the edges that connect the variable gadgets to the clause nodes.

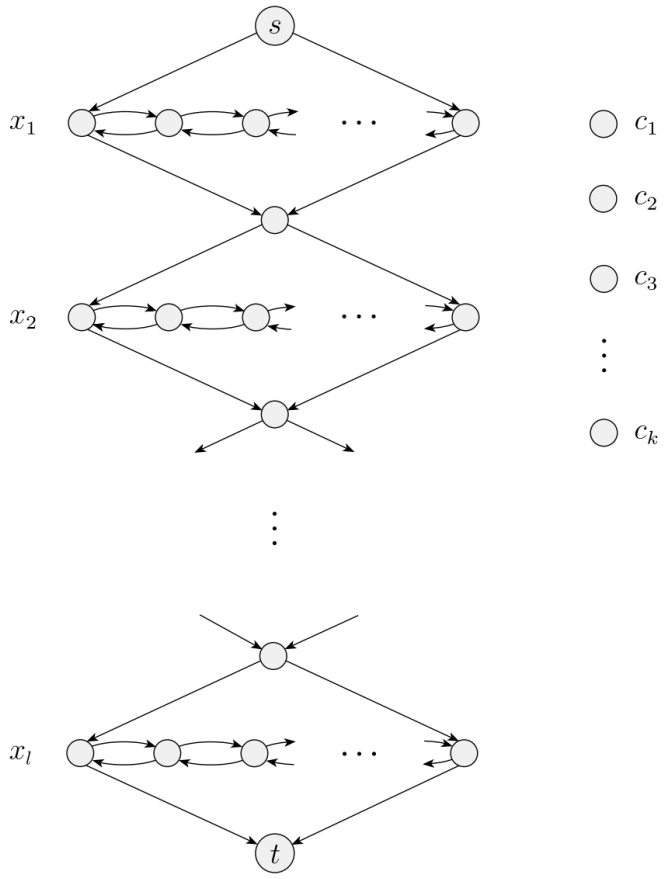

Figure 2: High-level structure of G.

Connecting variables to clauses. Each diamond's horizontal row contains $3k + 1$ nodes (in addition to the 2 tip nodes at the top and bottom of the diamond). These $3k + 1$ nodes are organized as follows: k adjacent **pairs** of nodes, one pair for each clause, separated by $k + 1$ **separator nodes** (one between each consecutive pair, plus one at each end of the row). The separator nodes act as buffers so that connections to different clause nodes do not interfere with each other. This gives $2k + (k + 1) = 3k + 1$ nodes in total.

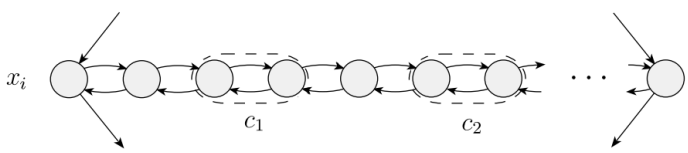

Figure 3: The horizontal row of a variable gadget, with dashed boxes indicating the pairs corresponding to clauses $c_1, c_2, \ldots$

The j-th pair in variable x_i's row serves as a potential detour point to clause node c_j. The detour edges are added as follows:

If variable x_i appears as a *positive* literal in clause c_j, we add two directed edges between the j-th pair in the i-th diamond and clause node c_j. These edges are oriented so that the detour through c_j is only possible when traversing left-to-right (the TRUE direction).

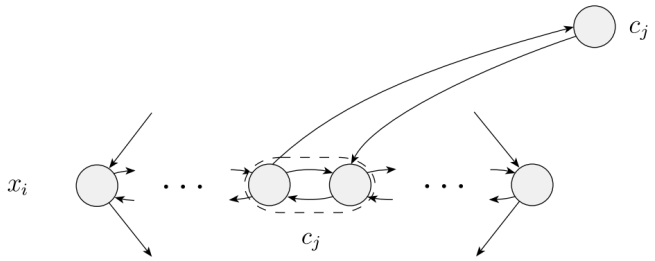

Figure 4: The additional edges when clause c_j contains x_i.

If $\overline{x_i}$ appears in clause c_j, we similarly add two directed edges between the j-th pair in the i-th diamond and clause node c_j, but oriented in the opposite direction so that the detour is only possible when traversing right-to-left (the FALSE direction).

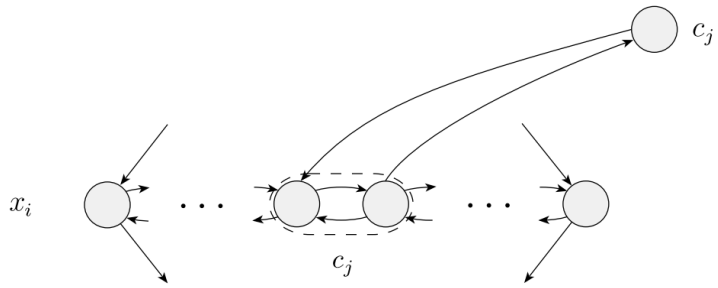

Figure 5: The additional edges when clause c_j contains $\overline{x_i}$.

If x_i does not appear in clause c_j at all, no edges are added, and the j-th pair in diamond i is simply traversed straight through.

Since each clause contains 3 literals from 3 distinct variables, each clause node c_j receives potential detour edges from exactly 3 different diamonds. In any Hamiltonian path, exactly one of those diamonds actually detours to c_j, since visiting it twice would violate the "every node exactly once" requirement. A single diamond, on the other hand, may detour to multiple different clause nodes along its row if the corresponding variable appears in multiple clauses.

Correctness.

($\Rightarrow$) *Satisfying assignment* $\implies$ *Hamiltonian path.*

Suppose ϕ has a satisfying assignment. We construct a Hamiltonian path from s to t as follows. The path begins at s and goes through each diamond in turn, ending at t. For each variable x_i, the satisfying assignment determines the direction: if $x_i =$ TRUE, the path zig-zags through the diamond from left to right; if $x_i =$ FALSE, it zag-zigs from right to left. That is, the top node of each diamond "chooses" a direction based on the truth value of x_i.

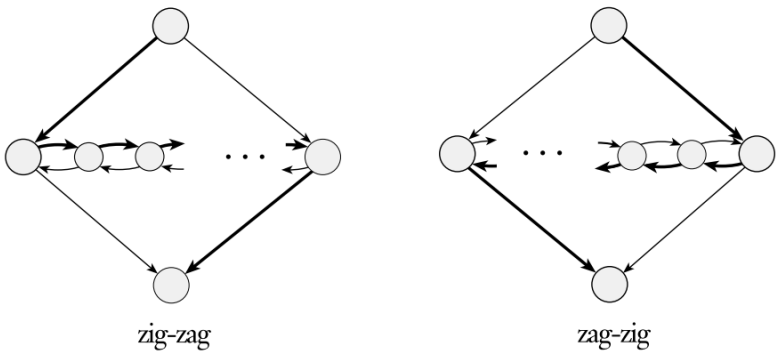

Figure 6: Zig-zagging (left-to-right, TRUE) and zag-zigging (right-to-left, FALSE) through a diamond, as determined by the satisfying assignment.

So far, this path covers all the nodes in G *except* the clause nodes. We include them by adding detours at the horizontal rows. For each clause c_j, since the assignment is satisfying, at least one literal in c_j is true. We select any one such true literal:

- If the selected literal is x_i, then $x_i = \text{TRUE}$, so the path zig-zags left-to-right through diamond i. The edges from the j-th pair to c_j are oriented to allow a detour in this direction, so the path can exit the row, visit c_j, and return.
- If the selected literal is $\overline{x_i}$, then $x_i = \text{FALSE}$, so the path zag-zigs right-to-left through diamond i. The edges from the j-th pair to c_j are oriented to allow a detour in this direction, so again the path can exit, visit c_j, and return.

Note that even if multiple literals in a clause are true, we take only one detour to c_j to avoid visiting it twice. Thus we have constructed the desired Hamiltonian path from s to t.

($\Leftarrow$) *Hamiltonian path* $\implies$ *satisfying assignment.*

Suppose G has a Hamiltonian path from s to t. We first show that the path must be *normal*—meaning it goes through the diamonds in order from top to bottom, only leaving the main chain to make detours to clause nodes. Once we establish normality, we can read off a satisfying assignment.

Reading off the assignment. The structure of the graph forces the path to traverse each diamond in a consistent direction—either entirely left-to-right or entirely right-to-left (with possible detours to clause nodes along the way). The separator nodes prevent the path from switching direction partway through a row. We define:

- $x_i = \text{TRUE}$ if diamond i is traversed left-to-right (zig-zag).
- $x_i = \text{FALSE}$ if diamond i is traversed right-to-left (zag-zig).

Why this assignment satisfies ϕ. The path is Hamiltonian, so it must visit every clause node c_j. The only way to reach c_j is via a detour from some variable x_i's row. Such a detour is only possible if the path traverses x_i's row in the direction corresponding to a literal that appears in clause c_j. Specifically:

- If the detour comes from a left-to-right traversal, then x_i appears positively in c_j and $x_i = \text{TRUE}$, so that literal is satisfied.
- If the detour comes from a right-to-left traversal, then $\overline{x_i}$ appears in c_j and $x_i = \text{FALSE}$, so that literal is satisfied.

In either case, clause c_j contains at least one true literal. Since this holds for every clause, ϕ is satisfied.

Proof that the path must be normal. Normality could fail only if the path enters a clause node c from one diamond but returns to a *different* diamond, as shown below.

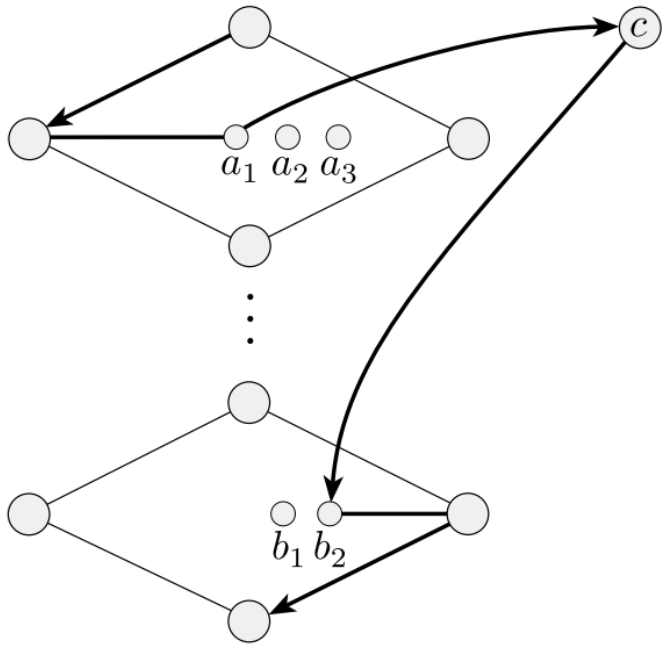

Figure 7: This situation cannot occur: the path goes from a_1 to c, but instead of returning to a_2 in the same diamond, it returns to b_2 in a different diamond.

Suppose this happens: the path goes from node a_1 to clause node c, then to b_2 in a different diamond instead of returning to a_2 in the same diamond. We show that node a_2 can never be visited, contradicting the assumption that the path is Hamiltonian.

The key observation is that a_2 has very few incoming edges (due to the constrained structure of the row), and every possible entry into a_2 is either already used by the path going elsewhere, or would force visiting another node twice. Concretely, consider two cases based on whether a_2 or a_3 is a separator node:

- **Case 1: a_2 is a separator node.** Separator nodes have only two incoming edges: from a_1 and from a_3. But a_1 already goes to c on the path, so a_1 cannot also go to a_2. The only remaining way to enter a_2 is from a_3. But a_2's only available outgoing neighbor is a_3 (separator nodes connect only to their immediate neighbors in the row), so the path would have to go $a_3 \to a_2 \to a_3$, which visits a_3 twice—a contradiction.
- **Case 2: a_3 is a separator node** (meaning a_1 and a_2 form a clause pair). Then the only edges entering a_2 come from a_1, a_3, and c. On the path, a_1 goes to c and c goes to b_2—both go elsewhere. The only remaining option is to enter a_2 from a_3. But again, a_3 is the only node that a_2 can exit to (since a_1 and c are already committed on the path), forcing $a_3 \to a_2 \to a_3$, which visits a_3 twice—a contradiction.

In both cases, the path cannot visit a_2, contradicting the Hamiltonian property. Therefore the path must be normal, completing the proof. □

2 UHAMPATH

We now consider the undirected version of the Hamiltonian path problem. To show that it is NP-complete, we give a polynomial-time reduction from the directed HAMPATH problem.

Remark 2.1: What are the instances?

- **Instance of** HAMPATH: A directed graph G with designated nodes s, t. The question: does a directed Hamiltonian path from s to t exist?
- **Instance of** UHAMPATH: An undirected graph G' with designated nodes s', t'. The question: does an undirected Hamiltonian path from s' to t' exist?

The reduction is a function f : directed graphs $\to$ undirected graphs, mapping $(G, s, t) \mapsto (G', s', t')$.

Remark 2.2: What are we building and why?

The challenge is: how do you simulate *direction* in an undirected graph? If you naively replace every directed edge $u \to v$ with an undirected edge $\{u, v\}$, a path could traverse an edge "backwards"—going from v to u even though the original only allowed $u \to v$. This would break the reduction: the undirected graph might have a Hamiltonian path even when the directed graph does not.

The solution is to replace each node u with a **triple** of nodes $(u^{\text{in}}, u^{\text{mid}}, u^{\text{out}})$. The middle node u^{mid} is connected *only* to u^{in} and u^{out}, so once you enter the triple from one side, you are forced through to the other. Directed edges $u \to v$ become undirected edges $\{u^{\text{out}}, v^{\text{in}}\}$. The starting point s' then triggers a chain reaction: it forces the first triple to be entered from the in side, which forces the second, which forces the third, and so on—effectively simulating directed traversal using only undirected edges.

Theorem 2.1

UHAMPATH is NP-complete.

Proof. UHAMPATH $\in$ NP by the same verifier as HAMPATH: the certificate is a sequence of nodes, and the verifier checks that it starts at s', ends at t', visits every node exactly once, and each consecutive pair is a valid edge. We now show HAMPATH $\leq_P$ UHAMPATH.

Construction. Given a directed graph G with designated nodes s and t, we construct an undirected graph G' with designated nodes s' and t' as follows.

Each node u in G (except s and t) is replaced by a **triple** of nodes $u^{\text{in}}, u^{\text{mid}}, u^{\text{out}}$ in G', connected by undirected edges:

$$u^{\text{in}} - u^{\text{mid}} - u^{\text{out}}$$

The node u^{mid} is connected *only* to u^{in} and u^{out}—this is the key constraint that will force consistent traversal direction.

The special nodes s and t are treated differently: s is replaced by a single node $s^{\text{out}} = s'$, and t is replaced by a single node $t^{\text{in}} = t'$. (Since s is only ever exited and t is only ever entered, they do not need full triples.)

For each directed edge $u \to v$ in G, we add an undirected edge $\{u^{\text{out}}, v^{\text{in}}\}$ in G'. (For edges out of s, this becomes $\{s', v^{\text{in}}\}$; for edges into t, this becomes $\{u^{\text{out}}, t'\}$.)

Correctness.

($\Rightarrow$) *Hamiltonian path in G $\implies$ Hamiltonian path in G'.*

Suppose G has a Hamiltonian path:

$$s \to u_1 \to u_2 \to \cdots \to u_m \to t$$

Then G' has the corresponding Hamiltonian path:

$$s',\ u_1^{\text{in}}, u_1^{\text{mid}}, u_1^{\text{out}},\ u_2^{\text{in}}, u_2^{\text{mid}}, u_2^{\text{out}},\ \cdots,\ u_m^{\text{in}}, u_m^{\text{mid}}, u_m^{\text{out}},\ t'$$

Each consecutive pair is connected by an edge in G' (either an internal triple edge or an edge corresponding to a directed edge in G), and every node in G' is visited exactly once.

($\Leftarrow$) *Hamiltonian path in G' $\implies$ Hamiltonian path in G.*

Suppose G' has a Hamiltonian path from s' to t'. We claim this path must traverse the triples in order: entering each triple through u^{in}, passing through u^{mid}, and exiting through u^{out}.

Note that a single triple alone does *not* prevent backward traversal—you could enter from u^{out}, pass through u^{mid}, and exit via u^{in}. What forces the correct direction is the **starting point combined with the chain structure**. We prove this by following the path step by step:

- The path starts at $s' = s^{\text{out}}$. The only nodes connected to s' are nodes of the form v^{in} (corresponding to edges $s \to v$ in G). So the path must move to some u_1^{in}.
- From u_1^{in}, the path must go to u_1^{mid}. Why? Because u_1^{mid} is only connected to u_1^{in} and u_1^{out}. If we don't visit u_1^{mid} now, the only other chance would be to enter from u_1^{out} later. But then u_1^{in} would be stranded—the path already left it, and there is no way to return without revisiting a node. So we must go to u_1^{mid} immediately.
- From u_1^{mid}, the only other neighbor is u_1^{out}, so the path goes there.
- From u_1^{out}, the only connections are to nodes of the form v^{in} (corresponding to edges $u_1 \to v$ in G). So the path moves to some u_2^{in}, and the argument repeats.

This chain reaction propagates from s' all the way to t': each triple is forced to be entered from the in side because the previous triple exited from its out side, and out nodes only connect to in nodes. Therefore the path must have the form:

$$s',\ u_1^{\text{in}}, u_1^{\text{mid}}, u_1^{\text{out}},\ u_2^{\text{in}}, u_2^{\text{mid}}, u_2^{\text{out}},\ \ldots,\ t'$$

which directly corresponds to a Hamiltonian path $s \to u_1 \to u_2 \to \cdots \to t$ in G. □

3 Bounded NFA Non-Universality

Definition 3.1: Bounded NFA Universality

For $k \geq 0$, we say that an NFA N is **k-universal** if it accepts all strings of length at most k. Conversely, N is **non-k-universal** if there exists a string of length at most k that N does not accept.
We define the following decision problem:

$$B\text{-}ALL_{\mathsf{NFA}} = \left\{ \langle N, 1^k \rangle \mid N \text{ is a } k\text{-universal NFA} \right\}$$

Note that k is encoded in **unary** (as the string 1^k). This is essential: in unary, writing 1^k requires k symbols, so k is at most linear in the input size. Had k been encoded in binary, only $\log k$ bits would suffice to represent it, making k potentially exponential in the input length. This would vastly increase the number of strings to enumerate (up to $|\Sigma|^k$), pushing the problem into a harder complexity class. Unary encoding ensures that k remains polynomially bounded in the input size, placing the problem precisely at coNP-complete.
In particular, $k \leq \ell$ where $\ell = |\langle N, 1^k \rangle|$ is the length of the input.

Theorem 3.1: $B\text{-}ALL_{\mathsf{NFA}}$ is coNP-complete

Recall that a language L is coNP-complete if and only if its complement $\overline{L}$ is NP-complete. Thus, it suffices to show that

$$\overline{B\text{-}ALL_{\mathsf{NFA}}} = \left\{ \langle N, 1^k \rangle \mid N \text{ is } \textbf{not } k\text{-universal} \right\}$$

is NP-complete.

Proof. **Step 1: $\overline{B\text{-}ALL_{\mathsf{NFA}}} \in \mathsf{NP}$.**

We construct a polynomial-time verifier V for $\overline{B\text{-}ALL_{\mathsf{NFA}}}$.

- **Input:** $\langle N, 1^k \rangle$.
- **Certificate:** A string $x \in \Sigma^*$ with $|x| \leq k$.
- **Verification:** V simulates N on x and accepts if and only if N **rejects** x.

Correctness: If $\langle N, 1^k \rangle \in \overline{B\text{-}ALL_{\mathsf{NFA}}}$, then N is not k-universal, so there exists some x with $|x| \leq k$ that N does not accept. This x serves as a valid certificate. Conversely, if no such x exists, no certificate will cause V to accept.

Running time: The certificate x has length at most k, and since k is encoded in unary, we have $|x| \leq k = O(n)$ where n is the input length. Simulating an NFA on x can be done by maintaining the set of reachable states at each step: starting from the ε-closure of q_0, for each symbol in x we compute the next set of reachable states. This takes $O(|x| \cdot |Q| \cdot |\delta|)$ time, which is polynomial in the input size. Hence V runs in polynomial time.

Remark 3.1: Why Unary Encoding Matters

Note that when encoding k in binary, the complexity of the verifier increases to exponential time. This is because for k given in binary, its value is exponential in the length of its encoding. The verifier runs in time that is at least linear in k (since even reading the input 1^k costs $O(k)$), which becomes exponential in the input length when k is given in binary.

Step 2: $\overline{B\text{-}ALL_{\mathsf{NFA}}}$ is NP-hard.

We reduce from UHAMPATH to $\overline{B\text{-}ALL_{\mathsf{NFA}}}$. Recall that UHAMPATH is NP-complete (proven previously via the start–mid–end triple technique).

Let $\langle G, s, t \rangle$ be an instance of UHAMPATH, where $G = (V, E)$ is an undirected graph with $n = |V|$. The reduction outputs $\langle N, 1^n \rangle$ with n encoded in unary, where N is an NFA constructed as follows.

Intuition. The NFA N is defined as the union of $n+1$ disjoint DFAs: $D_0, D_1, \ldots, D_n$, over the alphabet $\Sigma = V$. Since N is their union, it accepts a word if at least one D_i accepts it, and rejects a word only when *all* D_i reject it.

D_0 accepts exactly those words that do **not** describe a valid walk from s to t in G (i.e., it rejects only valid s-t walks). For each vertex $v_i \in V$, the DFA D_i accepts words in which v_i appears either zero times or at least twice (i.e., it rejects only words where v_i appears exactly once).

Thus, a word is rejected by all components—and hence rejected by N—precisely when it describes a valid s-t walk in which every vertex appears exactly once: a Hamiltonian path from s to t.

Formal construction. We set the alphabet to $\Sigma = V$.

For every $1 \leq i \leq n$, define the DFA $D_i = \langle Q^i, \Sigma, \delta_i, q_0^i, F^i \rangle$:

- $Q^i = \{q_0^i,\, q_1^i,\, q_2^i\}$.
- Initial state: q_0^i.
- Transitions: for $a \in \Sigma$,

$$\delta_i(q_0^i, a) = \begin{cases} q_1^i & \text{if } a = v_i \\ q_0^i & \text{otherwise} \end{cases}, \quad \delta_i(q_1^i, a) = \begin{cases} q_2^i & \text{if } a = v_i \\ q_1^i & \text{otherwise} \end{cases}, \quad \delta_i(q_2^i, a) = q_2^i \text{ for all } a.$$

- Accept states: $F^i = \{q_0^i,\, q_2^i\}$.

So D_i rejects x if and only if v_i appears exactly once in x.

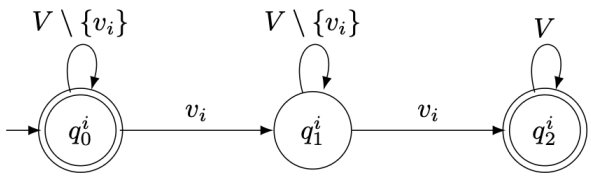

Figure 8: The DFA D_i: it accepts words in which v_i appears either zero times or at least twice.

Next, we define the DFA $D_0 = \langle Q^0, \Sigma, \delta_0, q_0^0, F^0 \rangle$ that accepts non-s-to-t paths. That is, every word that either does not start with s, does not end with t, or contains an invalid edge in G. Formally:

- $Q^0 = V \cup \{q_{\text{init}}, q_{\text{acc}}\}$, with initial state $q_0^0 = q_{\text{init}}$.
- $F^0 = Q^0 \setminus \{t\}$ (so ending in t is the only rejecting outcome).
- For $u, v \in V$:

$$\delta_0(u,v) = \begin{cases} v & \text{if } \{u,v\} \in E \\ q_{\text{acc}} & \text{if } \{u,v\} \notin E \end{cases}, \qquad \delta_0(q_{\text{acc}}, v) = q_{\text{acc}} \text{ for all } v, \qquad \delta_0(q_{\text{init}}, v) = \begin{cases} s & \text{if } v = s \\ q_{\text{acc}} & \text{if } v \neq s \end{cases}$$

D_0 simulates walking on G: the current state tracks the current vertex. It starts at q_{init} and moves to s upon reading s (or to q_{acc} if the first symbol is not s). From a vertex state u, reading v moves to v if $\{u,v\} \in E$, or to the absorbing accept state q_{acc} otherwise. The only rejecting state is t, so D_0 rejects exactly when the word describes a valid walk ending at t—i.e., a valid s-t walk. D_0 can be constructed in polynomial time since its $O(n)$ states and $O(n^2)$ transitions are read directly from G.

Finally, we form the NFA N as the union of these DFAs:

$$Q = \bigcup_{i=0}^{n} Q^i, \qquad Q_0 = \{q_0^i : 0 \leq i \leq n\}, \qquad F = \bigcup_{i=0}^{n} F^i,$$

and let δ agree with each δ_i on states from Q^i. This union is clearly computable in polynomial time.

Correctness. We claim that:

$$L(N) = V^* \setminus \{\, p \in V^* : p \text{ is a Hamiltonian path from } s \text{ to } t \text{ in } G \,\}. \tag{1}$$

In other words: N accepts every word **except** the Hamiltonian s-t paths in G. We first show that (1) implies the reduction is correct, and then prove (1).

(1) implies correctness:

($\Rightarrow$) Suppose $\langle G, s, t \rangle \in \mathsf{UHAMPATH}$. Then there exists a Hamiltonian path p from s to t in G. This path visits all n vertices, so $|p| = n$. By (1), $p \notin L(N)$, meaning N does not accept p. Since $|p| = n \leq n$, this witnesses that N is not n-universal. Hence $\langle N, 1^n \rangle \notin B\text{-}ALL_{\mathsf{NFA}}$, i.e., $\langle N, 1^n \rangle \in \overline{B\text{-}ALL_{\mathsf{NFA}}}$.

($\Leftarrow$) Suppose $\langle N, 1^n \rangle \in \overline{B\text{-}ALL_{\mathsf{NFA}}}$. Then N is not n-universal, so there exists a word p with $|p| \leq n$ that N does not accept. By (1), the only words not accepted by N are Hamiltonian paths from s to t in G. Therefore p is a Hamiltonian path from s to t in G, so $\langle G, s, t \rangle \in \mathsf{UHAMPATH}$.

Proof of (1): We need to show that N rejects $p \iff p$ is a Hamiltonian s-t path. Since N is the union of $D_0, D_1, \ldots, D_n$, we have: N rejects $p \iff$ *every* D_i rejects p.

($\Leftarrow$) Suppose p is a Hamiltonian s-t path. Then p is a valid walk from s to t, so D_0 rejects p. And every vertex v_i appears exactly once in p, so each D_i rejects p. Since all components reject p, N rejects p.

($\Rightarrow$) We prove the contrapositive: if p is **not** a Hamiltonian s-t path, then N accepts p. If p is not a Hamiltonian s-t path, it must fail in one of two ways:

- p is not a valid walk from s to t (it doesn't start at s, doesn't end at t, or uses a non-edge). Then D_0 accepts p, so N accepts p.
- p is a valid walk from s to t, but some vertex v_i appears $\neq 1$ times (either 0 or ≥ 2). Then D_i accepts p, so N accepts p.

In either case, at least one component accepts, so N accepts p.

This proves (1). Therefore the reduction is correct and runs in polynomial time, proving NP-hardness of $\overline{B\text{-}ALL_{\mathsf{NFA}}}$. □

Remark 3.2: The Unbounded Case

Later in the course we will see that the general universality problem for NFAs (i.e., deciding whether an NFA accepts all strings over its alphabet, with no bound k on the length) is even more difficult—it is PSPACE-complete.

Remark 3.3: Strategy: How to Approach NP/coNP-Completeness Reductions

When faced with proving that a language L is NP-complete or coNP-complete, follow this mental checklist:

1. **Match the quantifier.** Look at what L asks:
   - **Existential** ("does there exist...") → likely NP; reduce to L directly.
   - **Universal** ("for all...") → likely coNP; work with $\overline{L}$ instead, which flips the quantifier to existential.

   For example, $B\text{-}ALL_{\mathsf{NFA}}$ asks "does N accept *all* words of length $\leq k$?"—this is universal, hence coNP. Its complement asks "does there *exist* a word of length $\leq k$ that N rejects?"—this is existential, hence NP.
2. **Pick the source problem by structural similarity.** Ask: what known NP-complete problem has a similar shape?
   - Problem involves graphs or paths? Try HAMPATH or UHAMPATH.
   - Problem involves satisfying constraints? Try 3SAT.
   - Problem involves selecting subsets? Try CLIQUE, VERTEX-COVER, or SUBSET-SUM.

   For $\overline{B\text{-}ALL_{\mathsf{NFA}}}$: the rejected word is a sequence of vertices that looks like a path, so UHAMPATH is a natural fit.
3. **Think backwards: design the target instance so that solutions correspond.** Don't try to build the reduction forward. Instead ask: "What should the constructed instance look like so that yes-instances map to yes-instances?" In our case: "What NFA has the property that its rejected words are exactly the Hamiltonian paths?"
4. **Encode conjunctions via union (a recurring trick for automata problems).** If the property you need is a conjunction (condition 1 AND condition 2 AND ...), build separate automata that each accept *violations* of one condition, then take their union. The union rejects only when *all* conditions hold simultaneously.
5. **Watch the encoding.** Check whether numerical parameters are given in unary or binary—this affects the complexity class. Unary keeps the value polynomial in the input size; binary makes it exponential.